

Quantization for LLM Inference

pig7selene

September 5, 2026

Abstract

Quantization represents selected model tensors with fewer bits while retaining an approximation to their original values. This chapter develops affine integer quantization, its scale and zero-point parameters, and the distinctions among representation, granularity, and calibration. It then explains weight-only and weight-and-activation regimes, Post-Training Quantization and Quantization-Aware Training, activation outliers, and influential LLM methods including LLM.int8(), SmoothQuant, GPTQ, and AWQ. The emphasis is on the end-to-end trade-off among model memory, bandwidth, kernel execution, and output quality rather than on a particular inference library.

1 Introduction

Chapter 20 treated KV Cache quantization as a reduction in the persistent state created by active requests. This chapter concerns a different, mostly fixed object: the learned model weights loaded before requests arrive. A model with N parameters occupies approximately Nb bytes when each parameter is stored with b bytes. For sufficiently large models, that footprint can determine whether a checkpoint fits on an accelerator at all. Even when it fits, Decode often rereads a large fraction of those weights for every emitted token, so moving them through the memory hierarchy can limit throughput.

Quantization changes the representation used to store and execute selected tensors. It is not synonymous with low-precision floating-point arithmetic. FP16 and BF16 are floating-point formats with an exponent and significand; Chapter 8 explains their range and precision trade-offs. Integer quantization instead stores an integer code with an associated mapping back to an approximate real value. The mapping introduces error, but it can reduce weight capacity and memory traffic substantially. Whether it also improves latency depends on the execution kernel and hardware, not on the bit count alone.

The chapter uses a dense linear map as its running object. Let $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ be a weight matrix, $x \in \mathbb{R}^{d_{\text{in}}}$ an input activation, and $y = xW$ its output. The same representation questions apply to the projection matrices in attention and feed-forward layers. Cache-specific quantization remains distinct: it changes request-dependent keys and values, whereas this chapter primarily changes learned parameters and, in some regimes, transient activations.

2 Quantization as an Approximation Model

For a real-valued scalar x , affine quantization selects an integer code q through

$$q = \text{clip}_{q_{\min}, q_{\max}} \left(\text{round} \left(\frac{x}{s} \right) + z \right), \quad (1)$$

with approximate reconstruction

$$\hat{x} = s(q - z). \quad (2)$$

Here $s > 0$ is a **scale**, z is an integer **zero point**, and q lies in the representable code interval $[q_{\min}, q_{\max}]$. The scale sets the separation between adjacent reconstructed values; the zero point determines which code represents real zero. Rounding produces an irreducible representation error even when x lies in range. Clipping produces a larger error when x lies outside the interval that the selected scale can cover.

For an unsigned k -bit code, a conventional range is $q_{\min} = 0$ and $q_{\max} = 2^k - 1$. Given chosen real endpoints $x_{\min}$ and $x_{\max}$, a common affine calibration rule is

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \quad z = \text{round}\left(q_{\min} - \frac{x_{\min}}{s}\right). \quad (3)$$

The reconstructed grid covers the chosen interval only approximately because z must be integral. Its quality therefore depends on both the source distribution and the rule used to estimate the endpoints.

2.1 Symmetric and Asymmetric Schemes

Symmetric quantization centers the codebook at zero. For signed codes $q \in \{-Q, \dots, Q\}$, it commonly uses $z = 0$ and

$$q = \text{clip}_{-Q, Q}\left(\text{round}\left(\frac{x}{s}\right)\right), \quad s = \frac{\max|x|}{Q}. \quad (4)$$

It is simple to implement and maps positive and negative magnitudes symmetrically. It can, however, waste codes when the observed tensor is strongly shifted or has an asymmetric range. **Asymmetric quantization** uses the nonzero z in Equation 1 to place zero and the representable interval more flexibly. This can use a bounded nonnegative distribution efficiently, but it introduces zero-point metadata and can complicate some integer kernels.

Neither adjective states the whole quantization policy. A **datatype precision** specifies the code format, such as FP16, BF16, INT8, or INT4. A **quantization scheme** specifies the mapping, including signedness, symmetry, clipping, and rounding. A **granularity** specifies which elements share the same parameters s and z . A **calibration strategy** specifies how those parameters are selected. Confusing these axes leads to imprecise statements such as “the model uses INT4,” which says neither how its scale is shared nor which tensors still execute at a wider precision.

3 Granularity, Regimes, and Memory

A single scale for an entire tensor is inexpensive but must accommodate its widest range. Per-tensor quantization uses one pair (s, z) for all entries of W . Per-channel quantization assigns a pair to each output or input channel, according to the kernel’s convention. Group-wise quantization partitions a channel or flattened weight matrix into fixed-size groups and gives each group its own parameters. The groups are smaller than a whole tensor but usually larger than one scalar.

Granularity	Shared parameters	Primary advantage	Primary cost
Per-tensor	One s and z for all entries	Minimal metadata and simple packing	A single outlier can enlarge every step size
Per-channel	One pair per selected matrix channel	Adapts to channel-scale variation	More metadata and channel-aware kernels
Group-wise	One pair per fixed block of weights	A practical low-bit accuracy–metadata balance	Group layout, packing, and dequantization complexity

Table 1: Finer granularity restricts the range that each scale must cover, usually reducing quantization error. It also adds parameter metadata and constraints that an execution kernel must honor.

Finer sharing often improves accuracy because each scale represents a narrower local distribution. It is not free: the scale and zero point consume storage, group boundaries affect packing, and the kernel must fetch or apply the correct parameters. At very low bit widths, group-wise weight quantization is a common compromise between one global scale and costly per-element adaptation.

The ideal memory model makes the benefit visible. If a reference weight tensor has N values stored with b_{ref} bytes each, and a quantized representation stores k bits per code, then

$$M_{\text{ref}} = Nb_{\text{ref}}, \quad M_{\text{quant}} \approx N\frac{k}{8} + M_{\text{metadata}}. \quad (5)$$

Relative to FP16 weights, ideal INT8 storage is one-half and ideal INT4 storage is one-quarter before metadata, padding, or unquantized layers. Equation 5 applies only to the represented weights. It does not reduce activation buffers, temporary workspaces, or the KV Cache derived in Chapter 20. A deployment can therefore have a much smaller checkpoint while seeing a smaller reduction in total resident memory.

3.1 Weight-Only and Weight-and-Activation Quantization

In **weight-only quantization**, a low-bit representation replaces W , while the input activation x and the matrix-product accumulation remain in a wider type, commonly FP16 or BF16. A kernel can unpack and dequantize weight groups during the matrix multiplication rather than materializing an entire widened copy. This regime reduces the persistent parameter footprint and, when weights dominate memory reads, can reduce bandwidth demand. Its activations still require wide storage and arithmetic.

In **weight-and-activation quantization**, both W and selected activations have low-bit representations, often summarized as W8A8 for INT8 weights and activations. This can reduce traffic and enable integer-oriented matrix kernels more broadly, but activation quantization is harder. Activations are input-dependent, change by layer and token position, and may have rare large values. A static scale that works for one calibration set can clip another input; a dynamically computed scale adds work and may be difficult to fuse into a high-throughput kernel.

FP16 and BF16 remain useful baselines rather than failed forms of quantization. They preserve a floating exponent per value, so they have much wider dynamic range than a fixed-scale integer group. INT8 is often a moderate compression regime with relatively mature hardware support. INT4 gives larger capacity and bandwidth savings, but its coarse codebook makes granularity, calibration, and layer sensitivity more consequential. Mixed schemes retain higher precision for selected layers, channels, activations, or accumulations when a uniform format would create too much error.

4 Obtaining a Quantized Model

Post-Training Quantization (PTQ) starts from a completed floating-point checkpoint and chooses a representation without reoptimizing the original training objective. It may consist of a direct range-based conversion, a calibration pass through representative inputs, or a reconstruction procedure that minimizes the error of selected layer outputs. PTQ is attractive when retraining is unavailable or too expensive, but its result is limited by how well the calibration data and error model capture the deployment distribution.

Quantization-Aware Training (QAT) exposes the model to the approximate quantization path during optimization. A simplified forward model is

$$\widehat{W} = \text{dequant}(\text{quant}(W)), \quad y = x\widehat{W}, \quad (6)$$

while a surrogate gradient, often a Straight-Through Estimator, is used through the nondifferentiable rounding operation. The optimizer can then adapt weights to the quantization error. Foundational integer-only inference work combines quantized arithmetic with a co-designed training procedure [1]. QAT can improve accuracy at demanding precisions, but it reintroduces an optimization workflow, compute cost, and a risk that the learned tolerance is too specific to the simulated format or training distribution.

Calibration is a core PTQ contract, not an incidental preprocessing step. The selected examples determine observed ranges, activation statistics, and sometimes layerwise reconstruction objectives. They should cover the expected prompt domains, formatting, and sequence behavior of deployment. A calibration set consisting only of short plain-text prompts may omit code, structured inputs, multilingual text, or long-context patterns that produce different activation scales. More examples are not automatically better if their mixture is unrepresentative; the objective is coverage of the runtime tensor distributions relevant to the chosen quantization scheme.

5 Outliers and LLM Quantization Methods

An **outlier** is an unusually large-magnitude tensor value or channel relative to the bulk of its local distribution. In a group with one scale, a single large value increases s in Equation 1. The many ordinary values then occupy fewer distinguishable code levels and have larger relative rounding error. This is especially troublesome for activation quantization because activation ranges vary with the input. LLM.int8() identifies systematic Transformer feature outliers and combines vector-wise INT8 quantization with a mixed-precision path for the exceptional feature dimensions [2].

SmoothQuant addresses a related imbalance by applying an algebraically equivalent channel transformation before quantization. For a diagonal matrix D with positive entries,

$$y = xW = (xD^{-1})(DW) = x'W'. \quad (7)$$

The exact linear map is unchanged in real arithmetic. The choice of D moves channel scale between activations and weights, allowing activation outliers to be smoothed before an INT8 weight-and-activation path. SmoothQuant selects this migration from offline statistics and specifically targets the fact that activations can be more difficult to quantize than weights [3]. The transformation does not eliminate approximation error; it changes where the fixed quantization budget is spent.

5.1 GPTQ and AWQ

GPTQ is a one-shot PTQ method for low-bit weights. Rather than scoring an isolated weight error only by $|W - \widehat{W}|$, it uses calibration activations to approximate how a layer’s output changes. For a calibration activation matrix X , a representative reconstruction objective is

$$\min_{\widehat{W}} \|XW - X\widehat{W}\|_F^2. \quad (8)$$

The input correlation $X^T X$ supplies approximate second-order information about directions that the layer output is sensitive to. GPTQ quantizes weights sequentially and compensates remaining unquantized values using this information, aiming to limit output distortion at low bit widths [4]. The crucial point is not that every deployment must calculate a Hessian. It is that equal parameter-space errors are not equal functional errors once the activation distribution is considered.

AWQ is also a low-bit weight-only PTQ method, but it uses observed activation statistics to identify salient channels and chooses a hardware-friendly scaling that protects them. Its central premise is that the importance of a weight channel is better inferred from how it interacts with activations than from weight magnitude alone. The method uses offline statistics and avoids backpropagation or layer reconstruction in its stated workflow [5]. GPTQ and AWQ therefore make different approximations and impose different preprocessing and kernel expectations; neither name is a guarantee of identical quality across model families, group sizes, or workloads.

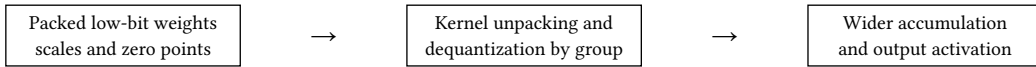


Figure 1: A weight-only kernel commonly combines unpacking, groupwise dequantization, and matrix multiplication. The stored weight representation and the accumulator precision are separate design choices.

6 Kernels and End-to-End Trade-offs

The stored bit width is not an end-to-end speed measurement. A low-bit weight may need to be unpacked, multiplied by a scale, and converted into a wider type inside a kernel, as illustrated in Figure 1. Those operations consume instructions and registers. On some devices they can be fused with matrix multiplication and replaced by efficient low-bit tensor operations; on others they can erase a theoretical bandwidth gain. An unsupported group size, layout, or signedness convention may force a fallback path that is slower than the FP16 baseline.

This creates a coupled trade-off. Lowering weight precision reduces M_{quant} in Equation 5 and can allow a larger model or more cache capacity to fit. It can also reduce memory bandwidth per Decode

step when the execution is weight-read dominated. But accuracy can fall from rounding, clipping, or accumulated layerwise error; latency can rise from dequantization; and arithmetic throughput depends on actual hardware and kernel support. The reference configuration must therefore state not only “INT4” or “INT8,” but also its weight and activation dtypes, quantization axes, group size, scale dtype, packing layout, accumulator type, and kernel backend.

Quantization error rarely distributes uniformly across a Transformer. An attention projection, an embedding or output head, or a feed-forward layer can be more sensitive than its neighbors. Repeated approximate layers can accumulate error, and a model that retains perplexity on a short validation corpus can still fail a long-context, code, or structured-generation workload. Evaluation should compare the quantized model with the exact reference under the intended prompt mixture, context lengths, decoding policy, and output-quality criteria. Averages are useful but do not replace inspection of sensitive tasks or failure tails.

6.1 Failure Modes

Table 2 separates common symptoms from their first diagnostic boundary. The remedies are deliberately conditional: replacing every scale or lowering every bit width indiscriminately can obscure the actual cause.

Observed failure	Likely mechanism	First check
Large quality drop in one layer or task	Layer sensitivity, clipping, or a poorly matched group scale	Compare layer outputs and task slices with a wider reference.
Failure on deployment prompts but not calibration prompts	Unrepresentative activation statistics or range selection	Audit calibration mixture, sequence lengths, and prompt formatting.
Erratic results around rare large features	Outlier amplification under shared scales	Inspect channelwise activation ranges and mixed-precision exceptions.
Smaller model footprint but no speedup	Dequantization, unpacking, or unsupported kernel dominates	Profile the selected kernel and verify the intended packed layout is used.
Quality degradation grows with depth	Repeated reconstruction error accumulates across layers	Measure layerwise output error and test a mixed-precision exception policy.

Table 2: Quantization failures arise from representation, calibration, model sensitivity, and execution. A smaller checkpoint alone does not identify which boundary is responsible.

7 Implementation Contracts

A quantized checkpoint requires a representation contract. For every quantized tensor or group, it must record code dtype, signedness, symmetry, scale and zero-point dtype, quantization axis, group size, clipping policy, packing order, and the expected accumulator precision. The loader must verify the model architecture and tensor shapes before interpreting packed bytes. A correct INT4 buffer with the wrong group orientation is not a slightly different approximation; it represents different weights. The calibration contract records the source model revision, preprocessing and Chat Template state, sample selection, sequence-length policy, activation collection points, and range or reconstruction objective. Reproducibility requires retaining the calibration seed and statistics, not only the final scale tensors. A deployment contract additionally declares the supported hardware and kernel path, including whether dequantization is fused and which layers intentionally remain at a wider precision. Validation should include a small deterministic set of prompts with reference logits or selected layer outputs, plus an evaluation suite that reflects the intended workload. Check both numerical deviation and task behavior. Measure model memory, allocator-visible memory, and latency separately; report Prefill and Decode behavior separately when they differ. Finally, distinguish a failed quality threshold

from a kernel fallback or unsupported format. These are different contract violations and call for different repairs.

8 Summary

Quantization maps real tensors to low-bit codes plus metadata and reconstructs an approximation through a scale and, when needed, a zero point. The representation is defined by more than INT8 or INT4: scheme, granularity, calibration, packing, accumulator precision, and execution kernel jointly determine its cost and error. Finer per-channel or group-wise scales can preserve local resolution, while their metadata and layouts increase systems complexity.

Weight-only quantization chiefly reduces persistent model memory and weight bandwidth; weight-and-activation quantization can extend the gain but faces input-dependent activation outliers. PTQ derives a new representation from a completed checkpoint and calibration data, whereas QAT optimizes through a simulated quantization path. LLM.int8(), SmoothQuant, GPTQ, and AWQ illustrate different ways to manage outliers and functional sensitivity rather than one universally optimal format. A useful inference configuration is therefore one whose memory saving, kernel behavior, quality boundary, and calibration provenance have all been measured under the actual deployment regime.

References

- [1] B. Jacob *et al.*, “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713. doi: 10.1109/CVPR.2018.00286.
- [2] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale,” in *Advances in Neural Information Processing Systems 35*, Curran Associates, Inc., 2022. doi: 10.48550/arXiv.2208.07339.
- [3] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models,” in *Proceedings of the 40th International Conference on Machine Learning*, in *Proceedings of Machine Learning Research*, vol. 202. PMLR, 2023, pp. 38087–38099. doi: 10.48550/arXiv.2211.10438.
- [4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” in *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2210.17323.
- [5] J. Lin *et al.*, “AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration,” in *Proceedings of Machine Learning and Systems*, 2024. doi: 10.48550/arXiv.2306.00978.